

prompt-clarify

Memory for who you are.

Context for what you're doing.

Product Blueprint

yash raj

Started 5 August 2026

github.com/Yash707077/prompt-clarify

Status: v0 in progress — 11.5 of 18 steps

prompt-clarify — Blueprint

The one-line pitch *Memory for who you are. Context for what you're doing.*

The one-sentence goal *Make the second prompt as good as the tenth — without the user doing anything.*

Start here

Note	What's in it
01 - The Problem	Why bad prompts happen, and why rewriting alone can't fix them
02 - How It Works	The core loop, with diagrams
03 - The Gap Detector	The seven dimensions and the "ask at most 3" rule
04 - Architecture	One engine, many surfaces

Market

Note	What's in it
05 - Competition	Who else is doing this, and how far ahead they are
06 - Competitor Teardown	Researched detail on Pretty Prompt and the three cracks in it

Building it

Note	What's in it
07 - The 18 Steps	The whole plan, with live progress
08 - Setup Guide	Accounts, installs, API keys
09 - Tools and Costs	Every tool, what it does, what it costs
10 - GitHub and Publishing	Repo, commits, npm, what makes a repo credible

What actually happened

Note	What's in it
11 - Step 0 Validation	The by-hand test that decided whether to build at all
12 - Progress Log	Day by day, what got done
13 - Code Reference	Every file, what it does, why it exists
14 - Bugs and Lessons	Every failure found, and what fixed it
15 - Open Problems	What is still wrong, honestly

Current state at a glance

Done: working CLI on the local machine. Asks five questions once, saves a profile, uses it to rewrite vague prompts. Colours, spinner, auto-copy. Published to GitHub.

Not done: npm publishing, README, tests, and the entire *context* half of the pitch.

Completion:

- v0 working locally — **~90%**
- Published for others to use — **~75%**
- Full vision including browser extension — **~30%**

Honest caveat: the positioning is "memory for who you are, context for what you're doing." Only the memory half exists. See 15 - Open Problems.

01 — The Problem

What people experience

They type a short, vague prompt. They get generic, unusable output. They conclude *"AI isn't that good."*

The actual root cause

The prompt was **underspecified**, not badly worded.

"Make me a marketing plan."

The model has no idea: what product, which audience, what budget, what channel, what format, what success looks like. So it averages across everything it has ever seen — and average output is what *generic* means.

Why existing fixes fall short

A rewriter that silently expands that prompt is **inventing** the missing details. The output *sounds* more confident but is built on guesses the user never approved.

This is not a hypothetical failure. It happened in our own testing — see 14 - Bugs and Lessons, failure #6, where the tool invented that a water bottle was "leak-proof and insulated" because it needed something to say.

The insight the product is built on

You cannot fix an underspecified prompt without getting the missing information from somewhere.

The information lives in the user's head, not in the text. Every tool in this category has to solve that. There are only three ways:

Approach	Who does it	Problem
Guess	Most one-click rewriters	Invents facts. Confident and wrong.
Ask every time	Pretty Prompt's Refine mode, LogicBalls	Accurate but slow. Friction on every single prompt.
Ask once, remember	This product	Requires building memory. Nobody has done it properly.

Who this is for

- **First:** developers and freelancers who use AI daily and have never learned prompt structure
- **Later:** everyone else, via added profiles rather than diluted questions

See 05 - Competition for why "for everyone" is the losing move at the start.

What "solved" looks like

The user types the same short prompt they always would. The tool silently fills in what it already knows. They get usable output on the first try, instead of five rounds of *"no, not like that."*

Time saved is the metric. Not prompt length.

02 — How It Works

The design decision that defines the product

Every competitor sits on one side of a trade-off:

- **Fast but guessing** — one-click rewrite, invents the missing details
- **Accurate but slow** — asks clarifying questions on every prompt

Neither is satisfying. Asking costs time; saving time is the whole point.

The third path: ask once during setup, then never again.

Day one: five questions, two minutes, once. Every day after: zero questions, zero friction, and it isn't guessing — it's recalling.

This is faster than Refine mode *and* more accurate than one-click mode. That is the entire competitive argument.

The loop

```
flowchart LR
    classDef general fill:#fff6b6,stroke:#af7e02
    classDef decision fill:#c6dcff,stroke:#305bab
    classDef terminator fill:#adf0c7,stroke:#087429
    classDef core fill:#dedaff,stroke:#6631d7

    n1([User types a vague prompt])::terminator
    n2{Is there a saved profile?}::decision
    n3[Run one-time setup: 5 questions]::core
    n4[Load profile from disk]::general
    n5[Decide which profile facts are RELEVANT]::core
    n6{Prompt already well specified?}::decision
    n7[Return almost unchanged]::general
    n8[Rewrite, filling only relevant gaps]::core
    n9[Print + copy to clipboard]::general
    n10([User pastes it into their AI])::terminator

    n1 --> n2
    n2 -->|No| n3
    n2 -->|Yes| n4
    n3 --> n4
    n4 --> n5
    n5 --> n6
    n6 -->|Yes| n7
    n6 -->|No| n8
    n7 --> n9
    n8 --> n9
    n9 --> n10
```

Why the "already well specified?" branch matters

It is not an optimisation. It is a correctness requirement.

A tool that rewrites *every* prompt is not judging — it is just generating. In testing, the tool took a prompt that already specified length, audience, tone and structure, and made it three times longer and worse.

Rewriting a good prompt is a failure, not a success. Restraint is the skill.

That sentence is now literally in the meta-prompt. See 13 - Code Reference.

What memory can and cannot fix

Tested against seven real prompts written by the user before the tool existed:

Prompt	Memory alone?
first we should make a blue print in miro	Yes
so tell me how much time it will take	Yes
add also a blueprint about where we start...	Yes
i want to add in github repos	Partly
can u mak this horizonatal	No

5 of 7 recoverable from memory alone.

Both failures needed **what the user was looking at**, not who the user is. Hence the two-part positioning:

Memory *for who you are.* **Context** *for what you're doing.*

The context half is not built. See 15 - Open Problems.

03 — The Gap Detector

This is the product. Everything else is packaging.

The seven dimensions

A prompt can be missing any of these. The engine checks each one.

#	Dimension	The question it answers
1	Goal	What outcome does the user actually want?
2	Audience	Who reads or uses the result?
3	Format	Essay, table, code, bullets, JSON? How long?
4	Constraints	Tone, budget, tech stack, word count, must-avoids
5	Context	Domain facts the model can't know
6	Examples	A sample of "good" — the highest-leverage input
7	Success criteria	How the user will judge the output

Rule 1 — Never fill more than three

If a gap can't be filled from memory and isn't critical, leave it. A prompt stuffed with seven filled dimensions is bloated, not better.

Rule 2 — Relevance before rewriting

The single biggest failure in testing came from applying *all* profile facts to *every* prompt.

What went wrong: the profile said "currently working on: a prompting tool called prompt-clarify." The prompt was "write a blog post about AI." Output: *"write a blog post about AI focusing on the prompting tool prompt-clarify."*

It changed the subject. That is the worst possible failure — worse than doing nothing.

The fix: the meta-prompt now runs a relevance pass *first*, with worked examples of what to discard:

```
Example: prompt is "write a blog post about AI".
- "Main audience: my clients" -> RELEVANT, sets who it is written for.
- "Preferred style: short"     -> RELEVANT, sets length and tone.
- "Working on: a CLI tool"     -> NOT RELEVANT. Adding it CHANGES THE TOPIC.
- "Uses AI for: writing code"   -> NOT RELEVANT to a blog post.
```

Rule 3 — Never invent

Two kinds of invention, both banned:

- **About the person** — only facts from the profile
- **About the subject** — if they say "a water bottle", do not decide it is insulated, leak-proof or durable

The second one was discovered in testing and was not in the original design. See 14 - Bugs and Lessons.

Rule 4 — No first-person words

`my clients` means nothing to an AI that has never met you. It must become `marketing clients`.

Same for `I`, `our`, `my project`.

Worked example

In: `write a blog post about AI`

Out:

Write a short and direct blog post about artificial intelligence aimed at marketing clients. Structure the post with concise sections and clear headings, keeping the overall length around 500 words.

Filled: audience, format, length, structure, tone. Correctly ignored: the user's current project, their coding habits.

Critical design note — checkboxes destroy memory

During Step 0 the user was given multi-select checkbox questions and **ticked every option**.

That is normal behaviour, not indecision. But a profile saying "*audience: everyone, pain: everything*" knows nothing.

The setup must use open questions asking for ONE answer. Never checkboxes.

This is implemented in `src/setup.ts` and is the reason the memory is specific enough to be useful.

04 — Architecture

The principle

One core engine. Many surfaces.

The CLI, the future npm library and the future browser extension all call the same code. Nothing about the product logic knows which surface it is running under.

```

flowchart LR
    classDef surface fill:#ccf4ff,stroke:#108ab3
    classDef core fill:#dedaff,stroke:#6631d7
    classDef data fill:#adf0c7,stroke:#087429
    classDef ext fill:#e7e7e7,stroke:#595959
    classDef later fill:#ffd8f4,stroke:#af3fb9

    s1[CLI - v0 BUILT]:::surface
    s2[npm library - v1]:::later
    s3[Browser extension - v2]:::later

    c1[index.ts - entry point]:::core
    c2[memory.ts - load and save profile]:::core
    c3[setup.ts - the 5 questions]:::core
    c4[fixer.ts - THE PRODUCT]:::core
    c5[ui.ts - colours, spinner, clipboard]:::core
    c6[llm.ts - the only file touching AI]:::core

    d1[(~/prompt-clarify/memory.json)]:::data
    e1[{{Google Gemini API}}]:::ext

    s1 --> c1
    s2 --> c1
    s3 --> c1

    c1 --> c2
    c1 --> c3
    c1 --> c4
    c1 --> c5
    c3 --> c2
    c2 --> d1
    c4 --> c6
    c6 --> e1
  
```

Why `llm.ts` is isolated

Every AI call goes through one function: `ask()`.

Nothing else in the project knows Gemini exists. If you switch to OpenAI, Claude or a local model, you change **one file** and nothing else breaks.

This already paid off twice:

1. When Google retired `gemini-2.5-flash` mid-build, the fix was one line
2. When we needed retry logic for rate limits, it went in one place and every caller benefited

Why memory lives in the home folder

`~/.prompt-clarify/memory.json` — **not** in the project folder.

Three reasons:

1. It survives if the project folder is deleted
2. It can never be committed to GitHub by accident
3. One profile works across every project the user has

Why zero dependencies for the UI

Colours are ANSI escape codes. Clipboard is `pbcopy` / `clip` / `xclip`, all built into the OS.

Adding `chalk` and `clipboardy` would mean two more packages, two more supply-chain risks, and a bigger install — for something that takes 40 lines.

Current dependency count: 2 (`@google/genai`, `dotenv`).

What is deliberately NOT built yet

Thing	Why not
Tests	Step 12
<code>bin</code> entry for <code>npx prompt-clarify</code>	Step 15
Context reading (screen / session)	See 15 - Open Problems
Multi-model support	Pick one, make it excellent
Hosted web app	Extension beats it on distribution

05 — Competition

The idea is not original. *Several products already do clarify-then-rewrite. That was discovered on day 1, after the blueprint was drawn. See 06 - Competitor Teardown for the researched detail.*

Why that is good news, not bad

An idea nobody has built usually means nobody wants it. Several teams independently converging on this problem is evidence the problem is real.

Ideas are not the scarce resource. Execution and distribution are. Dropbox was not the first file sync. Notion was not the first notes app.

What you cannot do is build the same thing with no differentiator and hope.

Who you're actually up against

- 1. The models themselves — the biggest threat** ChatGPT, Claude and Gemini all do quiet prompt-repair internally, improving every release. Any tool that only *polishes wording* gets absorbed within a year. → **Defence:** memory the model cannot have, because it does not persist across your sessions and does not control the UI.
- 2. Pretty Prompt — the leader** 40,000+ users, Chrome extension, MCP server, two Product Hunt #2 finishes. Their "Refine mode" is clarify-then-rewrite. → **Defence:** their clarifying questions are **paywalled at \$8.25/month**. A free, open-source tool that asks nothing and remembers everything attacks their differentiator directly.
- 3. One-shot rewriters (LogicBalls, SecondBrain)** Paste in, get a longer prompt out. Fast, but they hallucinate context. → **Defence:** you recall instead of guessing.
- 4. Doing nothing** Most people just re-prompt four times until it works. This is the real default, and it is free. → **Defence:** be faster than four rounds of retry. If your flow is slower, you lose. This is why memory-first matters more than questions.

The moat, ranked honestly

1. **Memory of the user** — compounds with use, cannot be copied
2. **Distribution** — living where prompts get typed
3. **Open source trust** — people will not paste confidential prompts into a black box
4. ~~The rewrite logic~~ — copyable in a weekend. Not a moat.

The positioning

Memory for who you are. Context for what you're doing.

Competitors have neither properly. Pretty Prompt's memory is a paid add-on bolted onto a per-prompt tool. Nobody reads session or screen context at all.

Why "for everyone" is the wrong first move

"For everyone" is exactly where Pretty Prompt already lives, with 40,000 users and paid distribution.

Choosing one group first does **not** limit who can use the tool. It only decides who you build for first, whose problems you fix first, and where you announce it.

- *"For everyone"* → setup questions must be vague enough to fit a novelist, a lawyer and an engineer. Vague questions produce vague memory produce mediocre output for all three.
- *"For developers first"* → ask about stack, codebase, testing. Memory is immediately useful. Then add a marketer profile. Then a writer profile. **Same engine, more profiles.**

You reach everyone by *sequence*, not by *dilution*.

06 — Competitor Teardown

Researched 5 August 2026. Re-check before launch — this market moves.

Pretty Prompt — the leader

prettyprompt.com · "Grammarly for Prompting"

Metric	Value
Users	40,000+
Chrome Store rating	4.9★ from 132 reviews
Product Hunt	#2 Product of the Day, twice (May 2025, Jan 2026)
Surfaces	Chrome extension, web app, MCP server
Pricing	Free 5/week → Pro \$8.25/mo → Team \$12.99/seat

Features: one-click improve, Refine mode (clarifying questions), prompt library, history, image-to-prompt, templates, per-use-case landing pages for 11 verticals including "Non-Native English".

Verdict: well-executed, 18+ months ahead. **Do not attack head-on.**

Others

Tool	What it does
LogicBalls	Free. Markets "refuses to guess", asks 1–2 questions
SecondBrain	Asks for goal, audience, context, constraints, format
Anthropic prompt improver	Free, built into the platform
prompt-ops (Meta), DSPy , promptfoo , TextGrad	Open source, but aimed at <i>developers optimising production prompts</i> , not everyday users

The three cracks

1. Clarifying questions are behind a paywall

"Refine Prompts (follow-up questions)" is a **Pro** feature — \$99/year.

The single most valuable mechanism in the category, the one that separates asking from guessing, costs money.

A free, open-source tool that never needs to ask undercuts their differentiator entirely.

2. Nothing serious in this category is open source for everyday users

Everything is closed and hosted. Every prompt you paste goes to someone's server. For anyone handling client or company information, that is disqualifying.

"Runs locally, your prompts never leave your machine" is a real, unclaimed position.

Note: currently *not fully true* — prompts go to Gemini. True local-first would need a local model. Worth being precise about in the README rather than overclaiming.

3. The developer surface is thin

Pretty Prompt has an MCP server, but the open-source tooling (DSPy, promptfoo) targets production prompt optimisation — not the "I'm about to type something vague into Claude Code" moment.

Wedge options considered

Option	Why it could work	Risk
A. Local-first / privacy CLI	Nobody owns "your prompts never leave your machine"	Small market; not yet true
B. Coding-agent clarifier	Vague prompts to Claude Code / Cursor waste real money. Pain is acute and measurable. Devs install CLI tools without friction.	Requires understanding dev workflows
C. Non-English-first	Pretty Prompt has a "Non-Native English" page but it's an afterthought	Needs real user research

Recommendation at the time: B, then A.

What actually happened: the user chose "for everyone", then in practice built for their own profile (marketing freelancer). This is the single biggest unresolved strategic question. See 15 - Open Problems.

07 — The 18 Steps

Who does what: yash does setup, judgement and testing. Claude writes the code.

The one rule: do the steps in order. Each step proves the last one worked. Skipping means debugging two unknowns at once.

PART 1 — Set up the computer COMPLETE (5 Aug)

- [x] **1. Install Node.js** — Node v22.23.1, npm 10.9.8
- [x] **2. Install VS Code** — already present
- [x] **3. Install Git** — Git 2.50.1, name/email configured
- [x] **4. GitHub account + repo** — Yash707077, repo `prompt-clarify` created public with MIT licence and Node `.gitignore`
- [x] **5. Free Gemini API key** — separate Google Cloud project, billing OFF

PART 2 — Build the tool

- [x] **6. Create project folder** (6 Aug) — cloned to `~/Desktop/prompt-clarify`
- [x] **7. Print hello in terminal** (6 Aug) — TypeScript + tsx running
- [x] **8. Connect to Gemini** (6 Aug) — first AI reply received
- [x] **9. Build MEMORY setup** (7 Aug) — `memory.ts`, `setup.ts`
- [x] **10. Build PROMPT FIXER** (7 Aug) — `fixer.ts`, the actual product
- [x] **11. Make it look nice** (7 Aug) — `ui.ts`, zero new dependencies
- [] **12. Test on 10 real prompts** *in progress* — runner built, blocked on daily quota

PART 3 — Publish

- [] **13. Push code to GitHub** — *(partially done; committing continuously)*
- [] **14. Write README with GIF**
- [] **15. Publish to npm** — needs a `bin` entry first
- [] **16. Show 5 real developers** — *the step that actually matters*

PART 4 — Later (week 6–8)

- [] **17. Build Chrome extension**
 - [] **18. Pay \$5 fee and publish** — do NOT pay early
-

Checkpoints

```

flowchart LR
    classDef check fill:#ffc6c6,stroke:#bd0909
    classDef step fill:#fff6b6,stroke:#af7e02

    s12[12. Test on 10 prompts]:::step
    g1{Does it run?}:::check
    g2{Are prompts actually better?}:::check
    s13[13-15. Publish]:::step
    s16[16. Show 5 developers]:::step
    g3{Does anyone use it?}:::check
    s17[17. Extension]:::step

    s12 --> g1
    g1 -->|No| s12
    g1 -->|Yes| g2
    g2 -->|No - rethink| s12
    g2 -->|Yes| s13
    s13 --> s16 --> g3
    g3 -->|No - fix first| s12
    g3 -->|Yes| s17

```

Those loops are the difference between a plan and a plan that admits it might be wrong.

Cost

Parts 1, 2 and 3 = ₹0. Only Step 18 costs money (\$5, one time, months away).

Time estimates — original vs actual

	Estimated	Actual
Part 1 (setup)	2 hours	~2 hours
Steps 6–8	2 days	1 session
Steps 9–11	4–5 days	1 session
Step 12	1 day	2+ sessions, blocked on quota

Lesson: writing code was faster than estimated. *Testing* was slower — because testing is where reality intrudes.

08 — Setup Guide

Everything needed before writing a line of code. Budget half a day. Completed 5 Aug 2026.

Accounts (all free)

Account	Why	Cost
GitHub	Where the code lives and gets published	Free
npm (npmjs.com)	Where people run <code>npm install</code> from	Free
Google AI Studio	The Gemini API key	Free tier, no card

Software

1. **Node.js** — lets JavaScript run on the computer instead of only in a browser. Installs `npm` too. Verify:

```
node -v
```

2. **VS Code** — the editor. Has a built-in terminal.

3. **Git** — tracks every change and pushes to GitHub. Verify: `git --version`

One-time Git config

```
git config --global user.name "yash raj"
git config --global user.email "raj800472@gmail.com"
```

The Gemini API key — read carefully

Get it at aistudio.google.com/apikey → **Create API key in new project.**

⚠ **Use a NEW project, not an existing one.** Free-tier quota is counted **per Google Cloud project**, not per account or per key. Extra keys in the same project add nothing — they all draw from the same pool. A separate project gets fresh quota.

⚠ **Never enable billing on that project.** Turning billing on permanently removes the free tier. Every call bills from the first token.

Quota, as of Aug 2026 (Google cut these 50–80% in Dec 2025 — verify before relying on them):

Model	Requests/day
gemini-2.5-pro	~100
gemini-flash-latest	~250
gemini-flash-lite-latest	~1,000

The API key rule

An API key is a password that costs money if leaked. People have had thousands of dollars stolen from keys pushed to public repos.

- Store it in `.env`
- `.env` must be in `.gitignore` before the first commit
- Never paste it into code, chat, or a screenshot

Verify it's protected:

```
grep -n "^\.env" .gitignore
```

If that prints nothing, stop and fix it.

GitHub authentication

GitHub stopped accepting passwords in 2021. Pushing needs a **personal access token**.

1. github.com/settings/tokens → Generate new token (classic)
2. Note: `prompt-clarify` , Expiration: 30 days
3. Scope: tick **public_repo** only (*least privilege — full repo grants more than needed for a public project*)
4. Copy the token — shown once
5. On `git push`: username = GitHub username, password = **the token**

The paste is invisible in Terminal. That is normal.

Current token expires 5 Sep 2026. Regenerate at the same URL when pushes start failing.

Done when

`node -v` works, VS Code opens, a key sits in `.env` , and an empty repo pushes successfully.

09 — Tools and Costs

Currently installed

Tool	What it does	Type
TypeScript	JavaScript that catches mistakes before running	dev
tsx	Runs TypeScript instantly, no build step	dev
@types/node	Type definitions for Node's built-ins	dev
@google/genai	Talks to Gemini in 3 lines instead of 30	runtime
dotenv	Loads the API key from <code>.env</code>	runtime

Runtime dependencies: 2. Deliberately small — every dependency is a supply-chain risk and a bigger install.

Deliberately NOT installed

Tool	Why not
<code>chalk</code>	ANSI colour codes are 8 lines of code
<code>clipboardy</code>	<code>pbcopy</code> / <code>clip</code> / <code>xclip</code> are built into every OS
<code>@inquirer/prompts</code>	Node's built-in <code>readline/promises</code> does the job
<code>zod</code>	Not needed while output is plain text, not JSON
<code>commander</code>	One argument does not need an argument parser

The trap: *beginners lose weeks configuring tools instead of building the product. Add a tool only when something actually hurts.*

Needed later

Stage	Tool	For
Step 12	Vitest	Running tests
Step 15	tsup	Bundling TypeScript into shippable JavaScript
Step 15	npm registry	Publishing
Optional	GitHub Actions	Running tests automatically on push
Step 17	WXT or Plasmo	Extension framework
Step 17	React + Tailwind	The popup UI

Costs — the honest total

Thing	Cost
Node, VS Code, Git, GitHub, npm	₹0 forever
Gemini API	₹0 — free tier, no card
Everything in Parts 1–3	₹0
Chrome Web Store (Step 18 only)	\$5 one-time

Do not pay the \$5 early. It is for Step 18, which is 6–8 weeks away. Money spent on an unfinished project makes it harder to quit if you should.

Model configuration

Set in `.env`, read by `src/llm.ts`:

```
GEMINI_API_KEY=your_key_here
# GEMINI_MODEL=gemini-flash-latest    # optional override
```

Context	Model	Why
Real usage	<code>gemini-flash-latest</code>	Better quality rewrites
Test runner	<code>gemini-flash-lite-latest</code>	4× quota; testing burns 10 requests a run

Tested side by side. Flash-lite gave visibly thinner rewrites — no structure, no length guidance. Quota was not worth the quality drop for normal use.

10 — GitHub and Publishing

Repo: <https://github.com/Yash707077/prompt-clarify> **npm name:** `prompt-clarify` — checked and available as of 5 Aug 2026

*Always check **npmjs.com** before naming a repo. Discovering the package name is taken after building is a painful rename.*

The daily loop — the only commands needed

```
git add .
git commit -m "what you changed"
git push
```

Plus `git status` when unsure what is happening.

Commit every time something works, not once a week. Small commits are how you recover when you break something.

Branches

Solo project: commit straight to `main`. Branches solve a coordination problem that does not exist yet. Add them when a second person joins.

Commit history so far

Commit	What
Connect to Gemini API	First working AI call
Fix licence mismatch, version, and package metadata	ISC → MIT, 1.0.0 → 0.1.0
Add memory system	<code>memory.ts</code> , <code>setup.ts</code>
Add prompt fixer - the core product	<code>fixer.ts</code>
Add colours, spinner and clipboard copy	<code>ui.ts</code>
Make model configurable, add retries, fix over-editing	Retry logic, model config

Publishing to npm (Step 15)

```
npm login
npm version 0.1.0
npm publish --access public
```

Before that works, `package.json` needs a `bin` entry so `npx prompt-clarify` runs. Currently only `npx tsx src/index.ts` works.

⚠ You cannot un-publish after 72 hours. Start at `0.1.0`, never `1.0.0`.

What makes a repo look credible

README is 90% of it. In order:

1. One sentence: what it does
2. **An animated GIF of it running** ← highest-impact single element
3. Install command
4. Before/after example of a real prompt it improved
5. How it works, briefly
6. Contributing + licence

Also: `LICENSE` (done — MIT), topics/tags (`prompt-engineering` , `cli` , `llm`), a repo description (done), and 3–5 issues tagged `good first issue`.

Security

- `.gitignore` must contain `.env` and `node_modules` **before the first push** verified
- Enable **secret scanning** in repo settings
- If a key is ever pushed: **revoke it immediately**. Deleting the commit is not enough — it is already in the history.

Getting the first users

Ship first, announce second. When v0 works: post to **r/LocalLLaMA**, **Hacker News (Show HN)**, and **Product Hunt**.

Lead with the before/after example, not the tech stack.

Milestones

- [x] Repo exists, `.gitignore` correct
- [x] First commit that runs
- [x] v0 works end-to-end on own prompts
- [] README with GIF
- [] Published to npm
- [] **First user who isn't you** ← the only one that really counts

11 — Step 0 Validation

Step 0 was a day with no code. *Test the idea by hand before building it. If nobody cares, three months are saved.*

What was tested

Can memory alone fix a vague prompt, with **zero questions asked**?

Test data: 7 real prompts yash wrote during the planning session — before the tool existed, with no idea they would be used as test data. Not invented, not cleaned up.

Result

5 of 7 fully fixable from memory alone. Verdict: PASS.

yash confirmed the rewritten prompts would have produced better answers.

Example

What was typed:

first we should make a blue print in miro

What memory alone produced:

Create a product blueprint in Miro for a memory-first prompting tool. Cover: the problem, user flow, architecture, competitors, roadmap. I'm new to coding, so keep it simple. Be direct.

Zero questions asked.

Finding 1 — Checkbox onboarding does not work

In setup, yash was given multi-select checkboxes and **ticked every option** on 2 of 4 questions.

That is normal user behaviour, not indecision. But *"audience: everyone, pain: everything"* is memory that knows nothing.

Design decision: the real product must use **open questions asking for ONE answer**, never multi-select.

Found in 4 minutes. Would have cost 3 weeks to discover in code.

This is now implemented in `src/setup.ts`.

Finding 2 — Two kinds of missing information

Prompt	Fixable by memory?
first we should make a blue print in miro	Yes
so tell me how much time it will take	Yes
add also a blueprint about where we start...	Yes
i want to add in github repos	Partly
can u mak this horizonatal	No

Both failures needed **what the user was looking at**, not who the user is.

This produced the positioning:

Memory for who you are. Context for what you're doing.

Competitors have neither properly. This is a sharper wedge than anything in 06 - Competitor Teardown.

Honest caveat

Sample size: 1 user, 7 prompts.

This proves the *mechanic* is worth building. It does **not** prove people will use it.

Step 16 — showing it to 5 real developers — is the step that actually validates the product. *Do not skip it.*

12 — Progress Log

Day 1 — 5 August 2026

Strategy and setup.

- Idea proposed: a tool that rewrites bad prompts
- **Pushed back on originality** — research found Pretty Prompt, LogicBalls, SecondBrain and Anthropic's own prompt improver already do clarify-then-rewrite. See 06 - Competitor Teardown
- Researched competitors properly. Found three cracks: paywalled questions, no open-source option, thin developer surface
- **Pivoted the idea:** from "ask questions every time" to "**ask once, remember forever**"
- Ran 11 - Step 0 Validation — 5 of 7 real prompts fixable from memory alone. **GO.**
- Discovered the checkbox problem and the memory/context split
- Steps 1–5 complete: Node, VS Code, Git, GitHub account, repo, API key

End state: 5 of 18. Repo exists, no code yet.

Day 2 — 6 August 2026

First working code.

- Step 6: repo cloned to `~/Desktop/prompt-clarify`
- Step 7: TypeScript running, first terminal output
- Step 8: **Gemini connected, first AI reply received**
- **Hit first real bug:** `gemini-2.5-flash` returned 404 — retired for new users mid-build. Fixed by switching to the `gemini-flash-latest` alias and writing `src/models.ts` to list what a key can actually access
- GitHub push required a personal access token (passwords deprecated since 2021)
- Fixed package metadata: licence said ISC while `LICENSE` said MIT; version said 1.0.0 at step 8 of 18

End state: 8 of 18. Working code on GitHub.

Day 3 — 7 August 2026

The product itself.

- Step 9: memory system built — `memory.ts`, `setup.ts`. Profile at `~/.prompt-clarify/memory.json`
- Step 10: **fixer.ts** — **the actual product**. First version was bad; see 14 - Bugs and Lessons
- Step 11: `ui.ts` — colours, spinner, clipboard. **Zero new dependencies**
- Step 12 started: built `testrun.ts` and a 10-prompt test set chosen to probe failure modes, not to flatter
- **Three test runs, three different failures:** over-editing, then 503 overload, then rate limits

- Model made configurable. Real usage on Flash, tests on Flash-lite
- Retry logic added for network errors, server overload and rate limits
- **Found the freelancer audience problem** — "my clients" vs "my clients' customers"
- Daily quota exhausted. Stopped.

End state: 11.5 of 18. Working product, half-tested.

Next session

1. Re-run the full 10-prompt test on fresh quota
2. Fill in human verdicts — **this cannot be automated**
3. Fix what it reveals
4. Steps 13–15: README, `bin` entry, npm publish

Cumulative

Day	Steps completed	Cumulative
1	Step 0 + Steps 1–5	5 / 18
2	Steps 6–8	8 / 18
3	Steps 9–11, half of 12	11.5 / 18

Three days from idea to working product. The estimate was 10 days for v0.

13 — Code Reference

Every file, what it does, why it exists. Total: ~450 lines.

File map

```
prompt-clarify/
├── src/
│   ├── index.ts      entry point – orchestrates everything
│   ├── memory.ts     load and save the user profile
│   ├── setup.ts      the one-time 5 questions
│   ├── fixer.ts      □ THE PRODUCT – rewrites the prompt
│   ├── llm.ts        the only file that touches the AI
│   ├── ui.ts         colours, spinner, clipboard
│   ├── models.ts     dev helper – lists available models
│   └── testrun.ts    dev helper – batch tests prompts
├── blueprint/        these notes
├── test-prompts.txt  the 10 test prompts
├── test-results.md   generated output for human judging
├── .env              API key – NEVER committed
├── .env.example      template for other people
├── .gitignore        blocks .env and node_modules
├── package.json
├── tsconfig.json
├── LICENSE           MIT
└── README.md         still 2 lines – Step 14
```

index.ts — entry point

The flow:

1. Read command-line arguments
2. Load memory — if none exists, run setup first
3. If no prompt given, show help + what it remembers
4. Otherwise: print BEFORE, start spinner, call the fixer, print AFTER, copy to clipboard

Commands:

```
npx tsx src/index.ts "your vague prompt"  # improve a prompt
npx tsx src/index.ts setup                # redo the 5 questions
npx tsx src/index.ts                      # show help + memory
```

memory.ts — the profile store

Location: `~/.prompt-clarify/memory.json` — home folder, not the project.

Why home: survives project deletion, can never be committed by accident, works across all projects.

Shape:

```
{
  "version": 1,
  "createdAt": "2026-08-07T...",
  "updatedAt": "2026-08-07T...",
  "profile": {
    "work": "marketing freelancer",
    "focus": "a prompting tool called prompt-clarify",
    "audience": "my clients' customers on social media",
    "commonTasks": "writing code",
    "style": "short and direct"
  }
}
```

Key functions: `loadMemory()`, `saveMemory()`, `profileToContext()`, `memoryPath()`

Design note: a corrupted memory file returns `null` rather than throwing. A broken profile should trigger re-setup, not a crash.

setup.ts — the one-time interview

Five open questions using Node's built-in `readline/promises` — no dependency.

#	Question	Captures
1	What do you do?	work
2	What are you working on right now?	focus
3	Who mainly reads or uses what you make?	audience
4	What do you use AI for most often?	commonTasks
5	How do you want AI answers written?	style

The rule enforced here: open text, ONE answer each. Never checkboxes. See 11 - Step 0 Validation Finding 1.

Empty answers are rejected and re-asked.

fixer.ts — THE PRODUCT

Builds a meta-prompt and sends it to the AI. The meta-prompt has three stages:

Step 1 — Decide what is relevant. Go through the profile line by line, discard anything that does not change how the answer should be written. Includes worked examples of what to drop.

Step 1b — Decide whether to rewrite at all. If the prompt already states audience, format, length and tone, leave it alone. *"Rewriting a good prompt is a FAILURE, not a success. Restraint is the skill."*

Step 2 — Rewrite. Only relevant gaps. No invention, about the person *or* the subject. No first-person words.

Output cleaning: strips code fences the model adds despite being told not to; throws on empty output.

llm.ts — the AI boundary

One function: `ask(prompt)` . Nothing else in the project knows Gemini exists.

Model: `process.env.GEMINI_MODEL ?? "gemini-flash-latest"`

Retry logic — up to 3 attempts:

Error type	Detected by	Wait
Network	<code>fetch failed</code> , <code>ECONNRESET</code> , <code>ETIMEDOUT</code> , <code>socket hang up</code>	2s, 4s
Server overload	<code>503</code> , <code>UNAVAILABLE</code> , <code>high demand</code>	2s, 4s
Rate limit	<code>429</code> , <code>RESOURCE_EXHAUSTED</code>	20s, 40s

Not retried: bad API key, missing model (404). Repeating those wastes the user's time.

Each error type gets a plain-English message instead of raw JSON.

ui.ts — presentation

Zero dependencies.

- **Colours** — raw ANSI codes. Respects `NO_COLOR` and disables when piped
- **Spinner** — braille frames at 80ms. Cleared with `\r\x1b[2K` (*padding with spaces leaves trailing whitespace — a real bug that was fixed*)
- **Clipboard** — `pbcopy` (mac), `clip` (win), `xclip` (linux). Never throws; failing to copy must not lose the result

models.ts and testrun.ts — dev helpers

`models.ts` lists every model a key can access. Written when Gemini retired a model mid-build. **Keep it** — this will happen again.

`testrun.ts` runs `test-prompts.txt` and writes before/after pairs to `test-results.md` . Forces flash-lite and 12s spacing.

It deliberately does not score anything. *A tool cannot mark its own homework.*

⚠ **Both would ship to npm users.** Exclude before Step 15.

14 — Bugs and Lessons

Every real failure found so far, and what fixed it. This is the most useful note in the vault — these are the things that would have been discovered painfully later.

1. The idea was not original

Found: Day 1, after the blueprint was drawn.

Pretty Prompt (40,000 users), LogicBalls, SecondBrain and Anthropic's own prompt improver already do clarify-then-rewrite.

Lesson: research competitors *before* designing, not after. The blueprint was drawn on assumptions that turned out to be wrong.

What saved it: the research also revealed the three cracks that became the actual strategy. See 06 - Competitor Teardown.

2. Checkboxes destroy memory

Found: Step 0, within 4 minutes.

Given multi-select questions, the user ticked **every option**. Memory reading "*audience: everyone, pain: everything*" knows nothing.

Fix: open questions asking for ONE answer.

Lesson: this cost 4 minutes to find by hand. It would have cost 3 weeks to find in code.

3. Google retired a model mid-build

Found: Step 8. `gemini-2.5-flash` returned 404 — "*no longer available to new users*".

Fix: use the `-latest` alias instead of a pinned version, and write `models.ts` to query what a key can actually access.

Lesson: never pin an exact model version. Vendors retire them without warning.

4. Licence contradiction

Found: during a review, not by any error.

`npm init -y` defaulted `package.json` to **ISC** while the `LICENSE` file said **MIT**. Two different answers about what people are allowed to do. Version also said `1.0.0` at step 8 of 18.

Fix: MIT everywhere, version `0.1.0`.

Lesson: defaults from generators are not decisions. Read what they wrote.

5. Topic drift — the worst failure

Found: Step 10, first test.

Profile said *"currently working on: a prompting tool called prompt-clarify."* Prompt was *"write a blog post about AI."* Output: *"write a blog post about AI focusing on the prompting tool prompt-clarify."*

It changed the subject of the request.

Fix: a relevance-filtering stage in the meta-prompt with worked examples of what to discard.

Lesson: *"use only facts from the profile"* was not enough. The model needed to be told explicitly that **most profile facts are irrelevant to most prompts**.

6. First-person leakage

Found: same test.

Output contained *"aimed at my clients"*. An AI reading that prompt has no idea who "my" is.

Fix: meta-prompt rule banning `my`, `I`, `our`. `my clients` → `marketing clients`.

7. Over-editing and inventing subject facts

Found: Step 12, test prompt #6.

Given an already-good prompt — *"Write a 200-word product description for a stainless steel water bottle, aimed at gym-goers, in a friendly tone, with three bullet points on features"* — the tool tripled its length **and invented product features that were never mentioned**: "temperature retention, durability, leak-proof seal."

Two failures at once: over-editing a good prompt, and inventing facts about the *subject*.

Fix: a "decide whether to rewrite at all" stage, plus an explicit ban on inventing subject facts.

Rewriting a good prompt is a failure, not a success. Restraint is the skill.

Lesson: the original "never invent" rule only covered facts about the *person*. It never occurred to anyone that the model would invent facts about the *product being described*.

8. No retry logic

Found: Step 12, run 1. A single dropped connection killed the whole run.

Fix: retries for network failures.

Then run 2 failed differently — four 503 "high demand" errors. The retry logic only covered network errors, not server overload.

Then run 3 failed differently again — eight rate limits.

Final fix: three categories, with 20s+ backoff for rate limits specifically.

Lesson: each fix revealed the next failure. Three runs, three completely different error classes.

9. Model choice is a quota decision, not just a quality one

Found: Step 12, run 3. Four test runs exhausted the daily quota.

`gemini-flash-latest` allows ~250 requests/day. `gemini-flash-lite-latest` allows ~1,000.

Fix: model configurable via `.env`. Real usage on Flash for quality; test runner forces Flash-lite.

Lesson: the `-latest` alias was chosen for resilience without considering that newer models have *tighter* free quotas. Two good properties in tension.

10. The freelancer audience problem

Found: Step 12.

For a marketing freelancer, "audience" is ambiguous. The user answered "*my clients*", so the tool wrote captions "*aimed at marketing clients*" — but an Instagram caption for a jewellery brand is read by the **client's customers**, not the client.

Current fix: reworded the profile answer to "*my clients' customers on social media*". **That is a patch, not a fix** — any other freelancer installing this hits the same confusion.

Proper fix: a sixth setup question separating *who commissions the work* from *who reads it*.

Status: △ open. See 15 - Open Problems.

11. Spinner left trailing whitespace

Found: Step 11, visually.

Clearing the spinner by writing spaces left trailing whitespace on the output line.

Fix: `\r\x1b[2K` — carriage return plus erase-line.

Lesson: small, but it would have shown up in the README GIF.

Meta-lesson

Writing the code was fast. Finding out it was wrong was slow.

Steps 9–11 took one session. Step 12 — testing — has taken longer than all the building combined, and found seven distinct problems.

Anyone who says a project is "nearly done" because the code is written has not tested it yet.

15 — Open Problems

An honest list of what is still wrong. Nothing here is hidden or softened.

Critical — affects the core pitch

1. The "context" half of the product does not exist

The positioning is:

Memory for who you are. **Context** for what you're doing.

Only memory is built.

11 - Step 0 Validation found that 2 of 7 real prompts could not be fixed by memory — they needed to know *what the user was looking at* (can u mak this horizonatal). That gap was identified as the sharper wedge, the thing no competitor does.

Right now the product is a memory-based prompt rewriter. Still differentiated from Pretty Prompt, but **it is the smaller half of the idea.**

Options: - Ship memory-only, get real feedback, add context later (*recommended — feedback beats building*) - Build context before publishing (*delays the only step that validates anything*)

Either way: do not describe the product as doing both until it does.

2. The target user is unresolved

Research recommended **developers first**, because they install CLI tools without friction and the GitHub repo becomes the distribution channel.

The user chose "**for everyone**", then in practice built for their own profile — a marketing freelancer.

Why it matters: the setup questions, the example prompts, and where it gets announced all depend on this. "For everyone" means vague questions, which means vague memory, which is exactly 11 - Step 0 Validation Finding 1 in a different form.

Unresolved.

Important — before publishing

3. Step 12 is not finished

The full 10-prompt test has never completed cleanly. **No human verdicts are filled in.**

This is the gate that decides whether the tool actually works. Everything after it is premature until it is done.

4. The freelancer audience problem is patched, not fixed

See 14 - Bugs and Lessons #10. The current fix is rewording one profile answer. Any other freelancer hits the same confusion.

Proper fix: a sixth question separating who commissions from who reads.

5. `npx prompt-clarify` does not work

No `bin` entry in `package.json`. Only `npx tsx src/index.ts` works — unacceptable for a published tool.

6. Dev helpers would ship to users

`models.ts` and `testrun.ts` are development tools. They must be excluded from the published package.

7. README is two lines

The repo currently looks abandoned to anyone who finds it.

8. `test-results.md` is committed full of errors

Generated output from a failed run is in the repo. Either regenerate it or add it to `.gitignore`.

Known gaps — scheduled

Gap	Scheduled for
No tests	Step 12
No CI	Optional, after Step 15
No <code>.env.example</code> guidance in README	Step 14
Only one AI provider	Deliberate — pick one, do it well

◦ Honest uncertainties — cannot be resolved by building

Will people actually use it? Adding a step before typing is friction. Memory reduces that friction more than competitors do — that is the bet. Nobody knows until Step 16.

Does a better prompt always mean a better answer? Usually. But sometimes the AI was fine and the user expected something it cannot do. This tool cannot fix that.

Is "runs locally, prompts never leave your machine" true? No. Prompts go to Gemini. Only the *memory* is local. Do not overclaim this in the README — it is the kind of thing that destroys trust when someone notices.

Can this beat Pretty Prompt? On the memory idea, plausibly. On polish, distribution and 40,000 existing users, no. Not yet, and not soon.

The single most important next action

Step 16 — show it to five real developers.

Not more features. Not the extension. Not a better README.

Every problem on this page except #1 and #2 is a small, fixable engineering task. The unresolved question is whether anyone wants this, and only real users answer that.